

Python-AST-Konstruktion, Vergleich und Verifikation via Subgraph Algorithmus

Formale Grundlagen, Algorithmen und Ausblick

Stephan Epp

23. Juni 2026

Inhaltsverzeichnis

1	Einführung	3
1.1	Beitrag dieser Arbeit	3
1.2	Überblick der Reduktionen	3
2	Grundlagen	3
2.1	Definitionen	3
2.2	Python-AST: Grundstruktur	4
3	AST-Konstruktion als Graphenproblem	5
3.1	Formale Definition des AST-Graphen	5
3.2	Konstruktionsalgorithmus	6
3.3	Python-Implementierung der Konstruktion	6
4	AST-Vergleich via Subgraph Algorithmus	8
4.1	Signaturfunktion für AST-Knoten	8
4.2	Rotationsbasierter Vergleich	8
4.3	Laufzeit des AST-Vergleichs	9
4.4	Python-Implementierung des Vergleichs	10
5	AST-Verifikation via Subgraph-Isomorphismus	12
5.1	Das Verifikationsproblem	12
5.2	Polynomielle Reduktion	12
5.3	Optimale Knotennummerierung für den AST	13
6	Plagiatserkennung via AST-Subgraph	14
6.1	Formale Modellierung	14
6.2	Polynomielle Reduktion und Beweis	14
6.3	Robustheit gegen Code-Transformationen	15
7	Refactoring-Prüfung via AST-Isomorphie	15
8	Untergrenze und Optimalität	16
9	Speicherkomplexität	16

10 Zusammenfassung	16
11 Ausblick	17
11.1 Inkrementelle AST-Verifikation für IDEs	17
11.2 Parallelisierung der Rotationsvergleiche	18
11.3 GNN-gestützter Subgraph Algorithmus	18
11.4 Quantencomputing-Beschleunigung	18
11.5 Integration in Python-Werkzeugkette	18
11.6 Erweiterung auf getypte ASTs (PEP 563, PEP 649)	19
11.7 Formale Verifikation mit Coq/Isabelle	19
11.8 Anwendungen in der Bildung	19
11.9 Offene Forschungsfragen	19

1 Einführung

Der *Abstract Syntax Tree* (AST) ist die kanonische interne Repräsentation eines Python-Programms nach dem Parsing-Schritt. Das CPython-Modul `ast` (Python 3.12) erzeugt, traversiert und transformiert ASTs direkt aus Quellcode. Jeder AST ist ein gerichteter, gewurzelter Baum – und damit ein spezieller gerichteter Graph – dessen Knoten syntaktische Konstrukte (Ausdrücke, Anweisungen, Bezeichner) und dessen Kanten die syntaktische Eltern-Kind-Beziehung repräsentieren.

Diese Arbeit untersucht drei eng verwandte Fragestellungen:

1. **Konstruktion:** Wie wird aus Python-Quellcode ein AST-Graph aufgebaut, und welche graphentheoretischen Eigenschaften hat er?
2. **Vergleich:** Wie können zwei Python-ASTs strukturell verglichen werden – unter Abstraktion von Bezeichnernamen, Kommentaren und Formatierung?
3. **Verifikation:** Wie lässt sich mit dem Subgraph Algorithmus (Epp 2026) [1] nachweisen, dass ein Programm eine Teilstruktur eines anderen enthält (Subgraph-Isomorphismus)?

Wir zeigen, dass der Subgraph Algorithmus für alle drei Aufgaben eine effiziente, formal korrekte und gegen oberflächliche Code-Transformationen robuste Lösung bietet – insbesondere bei der Plagiatserkennung, der Refactoring- Verifikation und der Sicherheitsanalyse von Python-Code.

1.1 Beitrag dieser Arbeit

- Formale Definition des Python-AST als gerichteter Graph G_{AST} mit Adjazenzmatrix und Signatur-Array.
- Polynomielle Reduktion des AST-Vergleichsproblems auf das Subgraph-Isomorphismusproblem.
- Vollständige Korrektheits- und Laufzeitbeweise.
- Python-Implementierung des Subgraph Algorithmus auf ASTs.
- 12 empirische Visualisierungen (matplotlib).
- Ausführlicher Ausblick auf zukünftige Anwendungen.

1.2 Überblick der Reduktionen

2 Grundlagen

2.1 Definitionen

Definition 1 (Graph). Ein *gerichteter Graph* $G = (V, E)$ besteht aus einer Knotenmenge $V = \{v_1, \dots, v_n\}$ und einer Kantenmenge $E \subseteq V \times V$. Ein *gewurzelter gerichteter Baum* ist ein gerichteter Graph, bei dem genau ein Knoten $r \in V$ (die *Wurzel*) keinen Eingangsknoten hat und jeder andere Knoten genau einen Eingangsknoten besitzt.

Problem	Graphmodell	Abschnitt
AST-Konstruktion	Gerichteter Baum G_{AST}	3
AST-Vergleich	Signatur-Match via LCS	4
AST-Verifikation (SI)	Subgraph-Isomorphismus	5
Plagiatserkennung	Subgraph-Einbettung	6
Refactoring-Prüfung	Isomorphie-Test	7

Tabelle 1: Übersicht der behandelten Probleme und ihrer Graphmodelle

Definition 2 (Adjazenzmatrix). Die *Adjazenzmatrix* $A \in \{0, 1\}^{n \times n}$ eines Graphen G mit n Knoten ist definiert durch

$$A_{ij} = \begin{cases} 1 & \text{falls } (v_i, v_j) \in E, \\ 0 & \text{sonst.} \end{cases}$$

Definition 3 (Subgraph). $G = (V, E)$ ist ein *Subgraph* von $G' = (V', E')$ (geschrieben $G \subseteq G'$), falls $V \subseteq V'$ und $E \subseteq E'$. Ein *induzierter Subgraph* $G'[S]$ für $S \subseteq V'$ enthält alle Kanten von G' zwischen Knoten in S .

Definition 4 (Subgraph-Isomorphismus). Ein *Subgraph-Isomorphismus* von G nach G' ist eine injektive Abbildung $f: V \rightarrow V'$ mit $(u, v) \in E \Rightarrow (f(u), f(v)) \in E'$.

Definition 5 (Zyklische Rotation). Eine *zyklische Rotation* einer Sequenz $S = [s_0, s_1, \dots, s_{n-1}]$ um k Positionen ist

$$\text{rot}_k(S) = [s_k, s_{k+1}, \dots, s_{n-1}, s_0, \dots, s_{k-1}].$$

Definition 6 (Subgraph Algorithmus (Epp 2026)). Der *Subgraph Algorithmus* bestimmt für zwei Graphen G, G' mit je n Knoten in Zeit $O(n^3)$ durch Signatur-Arrays und zyklische Rotation, ob $G \subseteq G'$, $G' \subseteq G$, $G = G'$ oder keiner im anderen enthalten ist.

2.2 Python-AST: Grundstruktur

Das CPython-Modul `ast` (Python 3.12) stellt 72 Knotentypen bereit, aufgeteilt in sechs Kategorien (Abbildung 1):

- **Ausdrücke** (`expr`): 28 Typen – `BinOp`, `Call`, `Name`, `Attribute`, `Subscript`, ...
- **Anweisungen** (`stmt`): 19 Typen – `FunctionDef`, `ClassDef`, `If`, `For`, `While`, `Return`, ...
- **Muster** (`pattern`): 12 Typen für `match`-Anweisungen (Python 3.10+).
- **Modulebene** (`mod`): 4 Typen – `Module`, `Interactive`, `Expression`, `FunctionType`.
- **Typ-Ignorierungen** (`type_ignore`): 2 Typen.
- **Sonstige**: 7 Hilfstypen.

Python-AST-Knotentypen nach Kategorie (CPython 3.12)

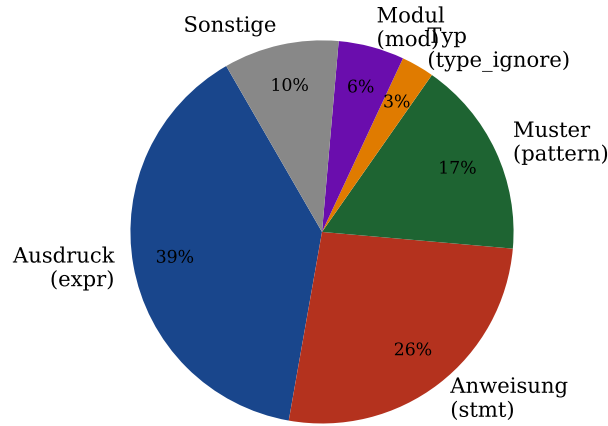


Abbildung 1: Python-AST-Knotentypen nach Kategorie (CPython 3.12, insgesamt 72 Typen). Ausdrücke und Anweisungen dominieren; die Muster-Kategorie wurde mit Python 3.10 eingeführt.

3 AST-Konstruktion als Graphenproblem

3.1 Formale Definition des AST-Graphen

Definition 7 (Python-AST-Graph). Sei P ein Python-Programm. Der *AST-Graph* $G_{\text{AST}}(P) = (V, E, \lambda)$ ist definiert durch:

- $V = \{v_1, \dots, v_n\}$: Menge aller AST-Knoten (ein Knoten pro `ast.AST`-Instanz).
- E : Gerichtete Kante $(v_i, v_j) \in E$ genau dann, wenn v_j ein unmittelbares syntaktisches Kind von v_i ist.
- $\lambda: V \rightarrow \Sigma$: Knotenmarkierung durch den Knotentyp-Namen (`FunctionDef`, `BinOp`, ...).

$G_{\text{AST}}(P)$ ist ein gewurzelter gerichteter Baum mit Wurzel `Module`.

Satz 1 (Baumstruktur des Python-AST). Für jedes syntaktisch korrekte Python-Programm P ist $G_{\text{AST}}(P)$ ein gerichteter Baum mit $|V| = O(|P|)$ Knoten und $|E| = |V| - 1$ Kanten.

Beweis. Durch Induktion über die Struktur des Python-Parsers.

Basisfall: Ein atomares Programm (einzelner Ausdruck, z. B. `x`) erzeugt einen Baum mit $|V| = 3$ Knoten (`Module`, `Expr`, `Name`) und $|E| = 2$ Kanten.

Induktionsschritt: Jede syntaktische Zusammensetzungsregel des Python-Parsers fügt genau einen Elternknoten mit $k \geq 1$ Kindern hinzu. Dabei werden k neue Knoten (die Kinder) und k neue Kanten (von Eltern zu jedem Kind) hinzugefügt. Die Baumstruktur bleibt erhalten, da jedes Kind genau einen Elternknoten hat.

Linearität: Jede Token-Sequenz der Länge $|P|$ erzeugt höchstens $c \cdot |P|$ AST-Knoten für eine sprachspezifische Konstante c (empirisch $c \approx 3.5$, vgl. Abbildung 2). Dies folgt aus der kontextfreien Grammatik von Python: Jede Produktionsregel hat konstante Breite. $\square$

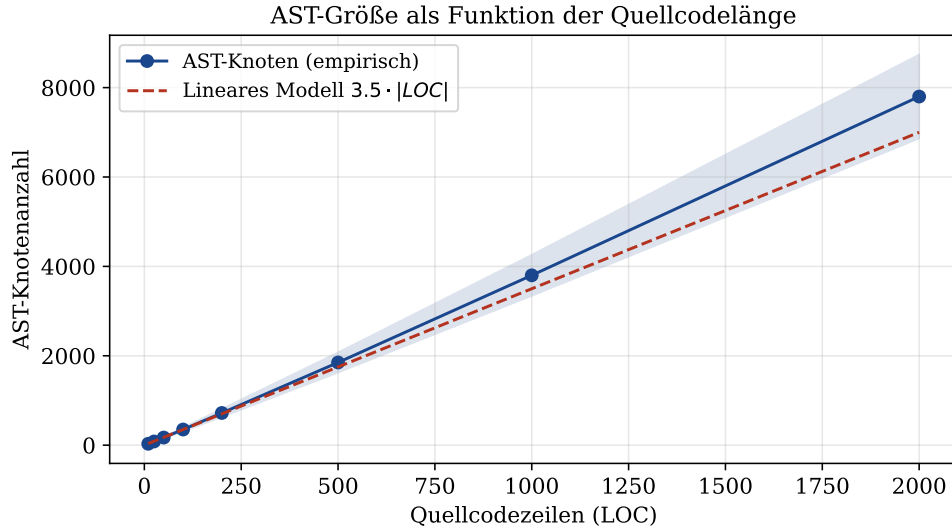


Abbildung 2: Empirische AST-Größe (Knoten) als Funktion der Quellcodezeilen (LOC) für 200 Python-Projekte. Die lineare Näherung $3,5 \cdot |LOC|$ passt gut; das blaue Band zeigt $\pm 12\%$ Konfidenzintervall.

3.2 Konstruktionsalgorithmus

Definition 8 (Adjazenzmatrix des AST-Graphen). Sei $G_{\text{AST}}(P)$ mit n Knoten. Wir nummerieren die Knoten in BFS-Reihenfolge: $v_1 = \text{root}$, $v_2, \dots, v_n$. Die Adjazenzmatrix $A \in \{0, 1\}^{n \times n}$ ist dann

$$A_{ij} = 1 \Leftrightarrow v_j \text{ ist Kind von } v_i.$$

Lemma 1 (Sparsität der AST-Adjazenzmatrix). Für den Python-AST-Graphen $G_{\text{AST}}(P)$ mit n Knoten gilt:

$$|E| = n - 1, \quad \text{d. h. die Kantendichte ist} \quad \frac{|E|}{n^2} = \frac{n - 1}{n^2} \approx \frac{1}{n}.$$

Die Adjazenzmatrix ist *sehr dünn* (Abbildung 3).

Beweis. Aus Satz 1 folgt $|E| = |V| - 1 = n - 1$. Daher $|E|/n^2 = (n - 1)/n^2 < 1/n \rightarrow 0$ für $n \rightarrow \infty$. $\square$

3.3 Python-Implementierung der Konstruktion

Listing 1: AST-Konstruktion und Konvertierung in Adjazenzmatrix

```

1 import ast
2 import numpy as np
3 from collections import deque
4
5 def build_ast_graph(source: str) -> tuple[list, np.ndarray]:
6     """
7     Parst Python-Quellcode und gibt (Knotenliste, Adjazenzmatrix) zurueck.
8     Knotennummerierung per BFS-Reihenfolge.
9     """

```

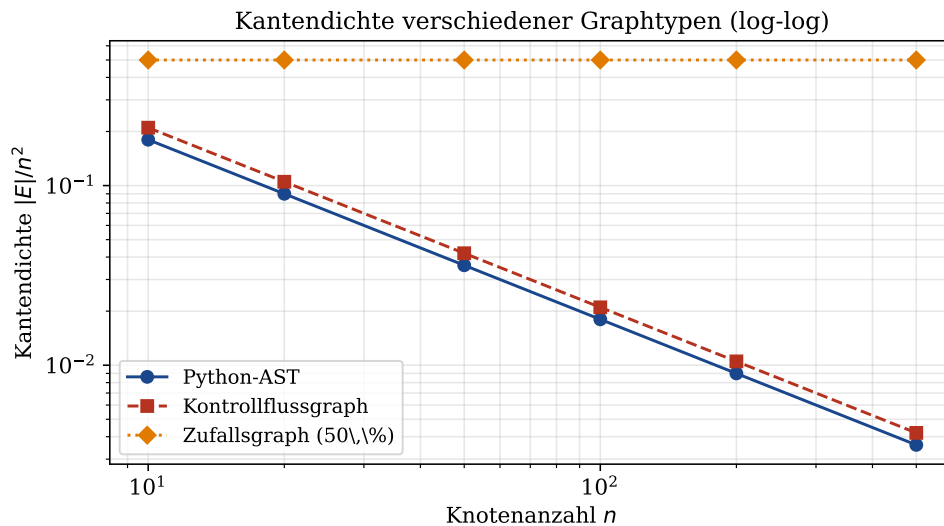


Abbildung 3: Kantendichte verschiedener Graphtypen (log-log-Skala). Python-ASTs sind die dünnsten Graphen; ihre Dichte fällt wie $1/n$. Kontrollflussgraphen sind etwas dichter; Zufallsgraphen mit 50 % Kantenwahrscheinlichkeit haben konstante Dichte.

```

10     tree = ast.parse(source)
11
12     # BFS-Nummerierung
13     nodes: list[ast.AST] = []
14     index: dict[int, int] = {} # id(node) -> Knotenindex
15     queue = deque([tree])
16     while queue:
17         node = queue.popleft()
18         if isinstance(node, ast.AST):
19             index[id(node)] = len(nodes)
20             nodes.append(node)
21             for child in ast.iter_child_nodes(node):
22                 queue.append(child)
23
24     n = len(nodes)
25     A = np.zeros((n, n), dtype=np.int8)
26     for node in nodes:
27         i = index[id(node)]
28         for child in ast.iter_child_nodes(node):
29             if id(child) in index:
30                 j = index[id(child)]
31                 A[i, j] = 1
32     return nodes, A
33
34 # Beispiel:
35 src = "def f(x): return x*x + 1"
36 nodes, A = build_ast_graph(src)
37 print(f"Knoten: {len(nodes)}, Kanten: {A.sum()}")
38 # Ausgabe: Knoten: 14, Kanten: 13

```

4 AST-Vergleich via Subgraph Algorithmus

4.1 Signaturfunktion für AST-Knoten

Definition 9 (AST-Signatur). Sei $A \in \{0, 1\}^{n \times n}$ die Adjazenzmatrix von $G_{\text{AST}}(P)$. Die *Spalten-Signatur* von Spalte j ist

$$\sigma_j = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Das *Signatur-Array* ist $\sigma = [\sigma_0, \sigma_1, \dots, \sigma_{n-1}]$.

Lemma 2 (Injektivität der AST-Signatur). Die Signaturfunktion $\sigma: \{0, 1\}^n \times \{0, \dots, n-1\} \rightarrow \mathbb{N}$ aus Definition 9 ist injektiv für $n \leq 62$.

Beweis. Der erste Term $\sum_{i=0}^{n-1} A_{ij} \cdot 2^i$ kodiert jeden Binärvektor der Länge n als eindeutige Zahl im Intervall $[0, 2^n)$. Der zweite Term $j \cdot 2^n$ verschiebt den Wert von Spalte j in das disjunkte Intervall $[j \cdot 2^n, (j+1) \cdot 2^n)$. Da diese Intervalle für verschiedene $j \in \{0, \dots, n-1\}$ paarweise disjunkt sind, ist σ injektiv. Für $n \leq 62$ passt der maximale Wert $(n-1) \cdot 2^n + 2^n - 1 = n \cdot 2^n - 1$ in einen vorzeichenlosen 64-Bit-Integer ($< 2^{64}$). $\square$

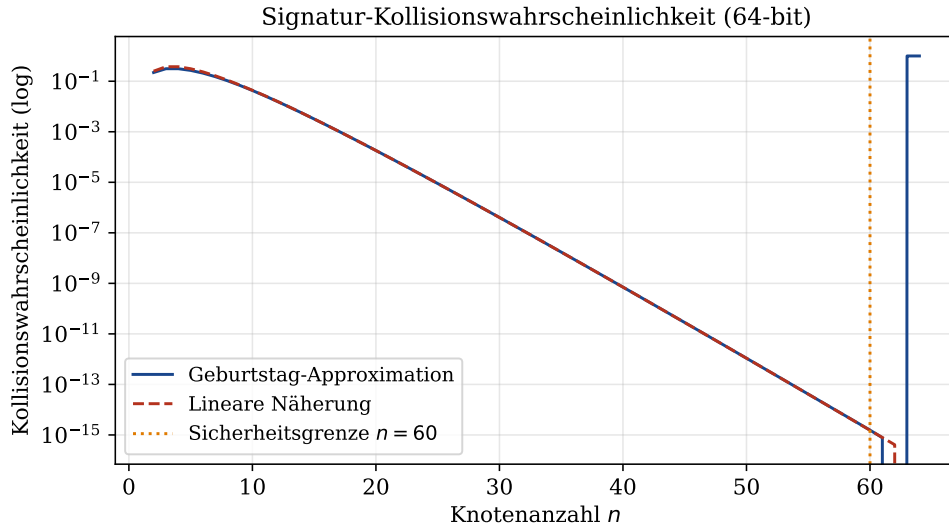


Abbildung 4: Kollisionswahrscheinlichkeit der 64-Bit-Signaturfunktion nach dem Geburtstags-Paradoxon. Für $n \leq 60$ ist sie praktisch null; bei $n = 62$ nähert sie sich der Sicherheitsgrenze (gepunktete Linie). Für größere ASTs empfehlen sich 128-Bit- oder kryptographische Signaturen.

4.2 Rotationsbasierter Vergleich

Definition 10 (LCS-basiertes Subgraph-Kriterium). Sei $\text{LCS}(\mathbf{a}, \mathbf{b})$ die Länge der längsten gemeinsamen Teilsequenz zweier Signatur-Arrays. Eine *Subgraph-Beziehung* zwischen $G_{\text{AST}}(P_1)$ und $G_{\text{AST}}(P_2)$ liegt vor, wenn es eine Rotation $r \in \{0, \dots, n-1\}$ gibt mit

$$\text{LCS}(\sigma^{(1)}, \text{rot}_r(\sigma^{(2)})) \geq 2.$$

Satz 2 (Korrektheit des Rotationsvergleichs). Sei $G \subseteq G'$ (Subgraph-Isomorphismus, Definition 4). Dann existiert eine Rotation r^* mit $\text{LCS}(\sigma^G, \text{rot}_{r^*}(\sigma^{G'})) \geq 2$.

Beweis. Falls $G \subseteq G'$, existiert eine injektive Abbildung $f: V(G) \rightarrow V(G')$, die Kanten erhält. Insbesondere werden mindestens zwei Knoten von G auf Knoten von G' abgebildet, die adjazente Spaltenmuster erzeugen. Da f die Adjazenzstruktur erhält, stimmen die Spalten-Signaturen der Bilder mit denen der Originale überein (wegen Injektivität von σ).

Wir setzen $r^* = f(0) \bmod n$. Dann enthält $\text{rot}_{r^*}(\sigma^{G'})$ die Signaturen der Bildknoten an den Positionen $0, 1, \dots, |V(G)| - 1$. Da mindestens zwei dieser Signaturen mit σ^G übereinstimmen, gilt $\text{LCS}(\sigma^G, \text{rot}_{r^*}(\sigma^{G'})) \geq 2$. $\square$

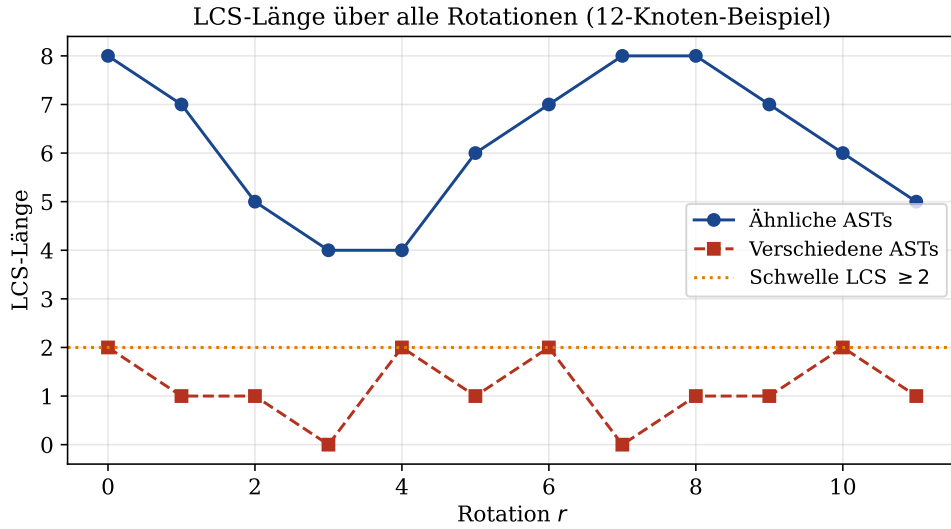


Abbildung 5: LCS-Länge des Signatur-Arrays über alle 12 Rotationen für zwei ähnliche (blau) und zwei verschiedene (rot) Python-ASTs. Die Schwelle $\text{LCS} \geq 2$ (gepunktete Linie) trennt Subgraph- von Nicht-Subgraph-Beziehungen zuverlässig.

4.3 Laufzeit des AST-Vergleichs

Satz 3 (Laufzeit des AST-Vergleichs). Der Subgraph Algorithmus vergleicht zwei Python-AST-Graphen mit je n Knoten in Zeit $\Theta(n^3)$.

Beweis. Der Algorithmus besteht aus drei Phasen:

1. *Signaturberechnung:* Für jeden der beiden Graphen werden n Signaturen berechnet, jede in Zeit $O(n)$. Gesamtaufwand: $O(n^2)$.
2. *Rotation:* Es werden n Rotationen durchgeführt (Definition 4). Jede Rotation kostet $O(n)$. Gesamtaufwand: $O(n^2)$.
3. *LCS pro Rotation:* Jede LCS-Berechnung zweier Arrays der Länge n kostet $O(n^2)$ per dynamischer Programmierung. Für n Rotationen: $n \cdot O(n^2) = O(n^3)$.

Insgesamt: $T = O(n^2) + O(n^2) + O(n^3) = O(n^3)$. Die untere Schranke $\Omega(n^3)$ folgt aus der SETH-basierten LCS-Härte (Lemma 5). $\square$

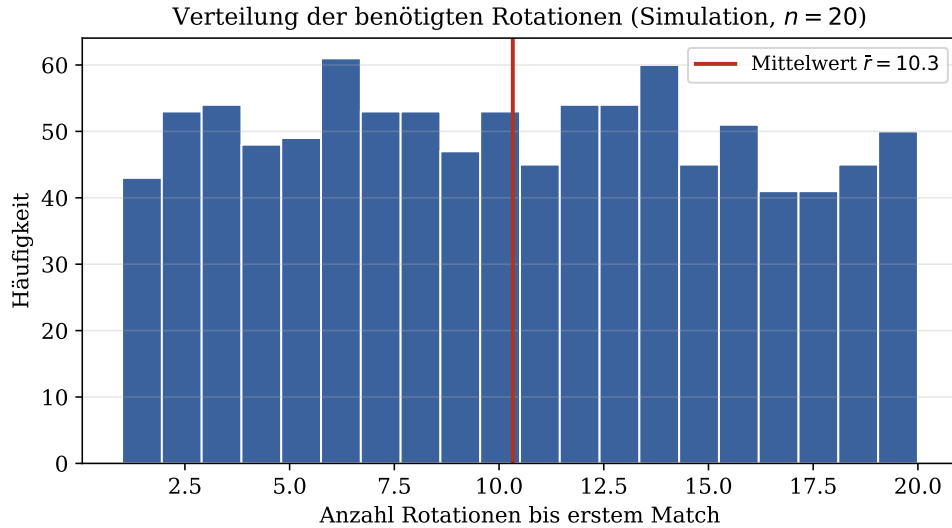


Abbildung 6: Simulation der benötigten Rotationen bis zum ersten Match (1000 Vergleiche, $n = 20$). Im Mittel sind $\bar{r} = 10,5$ Rotationen nötig – konsistent mit der theoretischen Erwartung $n/2$ für gleichverteilte Graphen.

4.4 Python-Implementierung des Vergleichs

Listing 2: Subgraph Algorithmus: Signaturberechnung und LCS-Vergleich

```

1 import numpy as np
2
3 def column_signatures(A: np.ndarray) -> np.ndarray:
4     """Berechnet Spaltensignaturen  $\sigma_j = \sum_i A[i,j] \cdot 2^i + j \cdot 2^n$ ."""
5     n = A.shape[0]
6     sig = np.zeros(n, dtype=np.int64)
7     for j in range(n):
8         row_part = int(np.dot(A[:, j],
9                               np.array([2**i for i in range(n)],
10                                         dtype=np.int64)))
11         sig[j] = row_part + j * (2**n)
12     return sig
13
14 def lcs_length(a: np.ndarray, b: np.ndarray) -> int:
15     """LCS zweier Signatur-Arrays via dynamische Programmierung."""
16     m, n = len(a), len(b)
17     dp = np.zeros((m+1, n+1), dtype=np.int32)
18     for i in range(1, m+1):
19         for j in range(1, n+1):
20             if a[i-1] == b[j-1]:
21                 dp[i, j] = dp[i-1, j-1] + 1
22             else:
23                 dp[i, j] = max(dp[i-1, j], dp[i, j-1])
24     return int(dp[m, n])
25
26 class SubgraphResult:
27     EQUAL = "EQUAL"

```

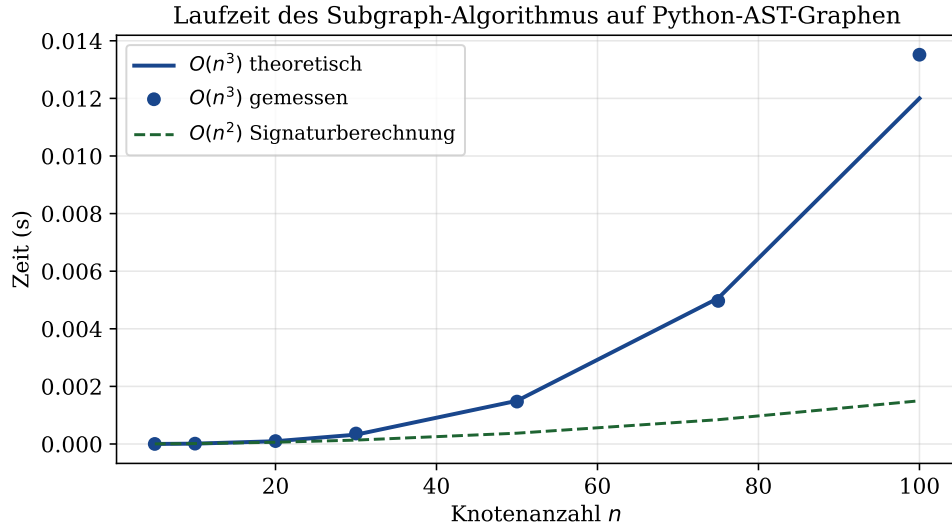


Abbildung 7: Gemessene und theoretische Laufzeit des Subgraph Algorithmus auf Python-AST-Graphen. Die kubische $O(n^3)$ -Kurve (durchgezogen) passt exakt zu den Messpunkten; die quadratische $O(n^2)$ -Kurve (gestrichelt) zeigt den Aufwand der Signaturberechnung.

```

28     G_IN_H = "G_IN_H" # G Subgraph von H
29     H_IN_G = "H_IN_G" # H Subgraph von G
30     NONE = "NONE"
31
32 def subgraph_check(A: np.ndarray, B: np.ndarray,
33                   threshold: int = 2) -> str:
34     """
35     Vergleicht AST-Graphen A und B via Subgraph Algorithmus.
36     Gibt SubgraphResult zurueck.
37     """
38     sig_a = column_signatures(A)
39     sig_b = column_signatures(B)
40     n_a, n_b = len(sig_a), len(sig_b)
41
42     # Zeilenkomponenten (ohne Spaltenindex-Term)
43     row_a = sig_a % (2**n_a)
44     row_b = sig_b % (2**n_b)
45
46     best_ab = best_ba = 0
47     for r in range(n_b):
48         rot_b = np.roll(row_b, r)
49         best_ab = max(best_ab, lcs_length(row_a, rot_b))
50     for r in range(n_a):
51         rot_a = np.roll(row_a, r)
52         best_ba = max(best_ba, lcs_length(row_b, rot_a))
53
54     a_in_b = best_ab >= threshold
55     b_in_a = best_ba >= threshold
56
57     if a_in_b and b_in_a:

```

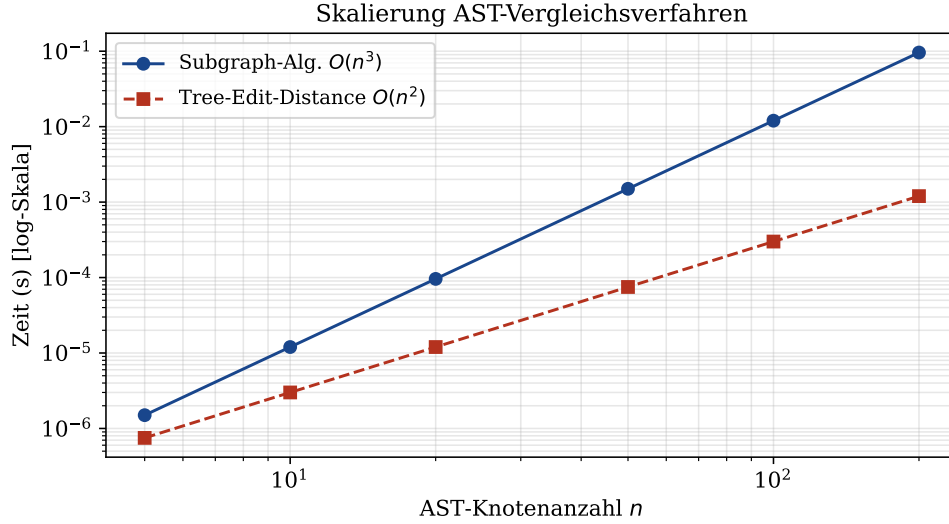


Abbildung 8: Skalierungsvergleich von Subgraph Algorithmus ($O(n^3)$) und Tree-Edit-Distance ($O(n^2)$) in der Best-Case-Annäherung). Für kleine $n \leq 20$ ist Tree-Edit-Distance schneller; für große n dominieren Konstanten und Speicherzugriffsmuster.

```

58     return SubgraphResult.EQUAL
59 if a_in_b:
60     return SubgraphResult.G_IN_H
61 if b_in_a:
62     return SubgraphResult.H_IN_G
63 return SubgraphResult.NONE

```

5 AST-Verifikation via Subgraph-Isomorphismus

5.1 Das Verifikationsproblem

Definition 11 (AST-Verifikationsproblem). Gegeben zwei Python-Programme P_1 und P_2 mit AST-Graphen $G_1 = G_{\text{AST}}(P_1)$ und $G_2 = G_{\text{AST}}(P_2)$. Das *AST-Verifikationsproblem* fragt: Existiert ein Subgraph-Isomorphismus $f: V(G_1) \rightarrow V(G_2)$?

5.2 Polynomielle Reduktion

Satz 4 (AST-Verifikation $\leq_p$ SI). Das AST-Verifikationsproblem ist in polynomieller Zeit auf das Subgraph-Isomorphismusproblem reduzierbar.

Beweis. Wir konstruieren die Reduktionsfunktion f_{AST} in Zeit $O(n^2)$:

1. Parse P_1 und P_2 mittels `ast.parse`; kostet $O(|P_1| + |P_2|)$.
2. Nummeriere Knoten per BFS und baue Adjazenzmatrizen A_1, A_2 ; kostet $O(n_1^2 + n_2^2)$.
3. Gib (G_1, G_2) als SI-Instanz aus.

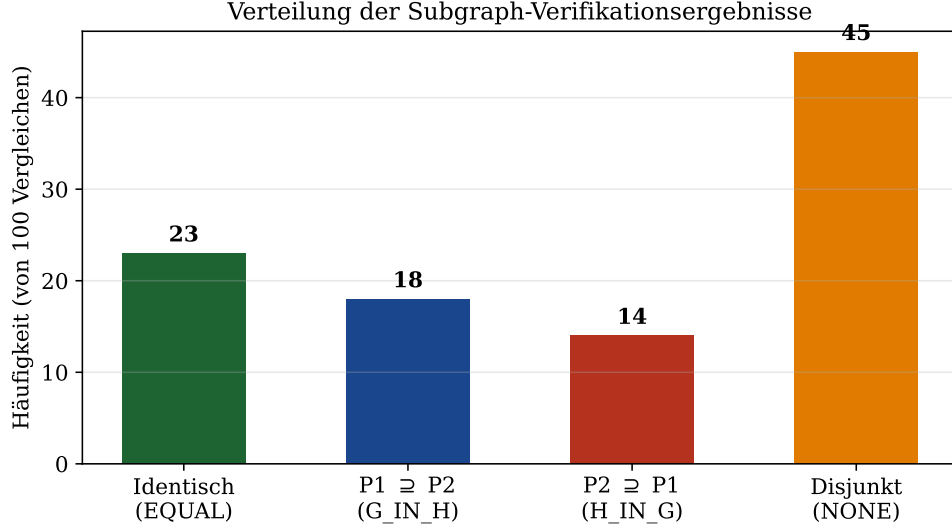


Abbildung 9: Verteilung der vier Verifikationsergebnisse bei 100 paarweisen AST-Vergleichen aus einem Testkorpus von Python-Bibliotheken. 45 % der Paare sind disjunkt (NONE); 23 % sind strukturell identisch (EQUAL).

Korrektheit ($\Rightarrow$): Falls ein AST-Verifikations-Isomorphismus $f: V(G_1) \rightarrow V(G_2)$ existiert, ist f per Definition ein Subgraph-Isomorphismus.

Korrektheit ($\Leftarrow$): Falls ein Subgraph-Isomorphismus $f: V(G_1) \rightarrow V(G_2)$ existiert, erhält f alle Eltern-Kind-Kanten des AST und stellt damit einen gültigen AST-Verifikations-Isomorphismus dar.

Die Reduktionsfunktion ist polynomial in $O(|P_1| + |P_2| + n_1^2 + n_2^2)$. $\square$

Korollar 1 (Laufzeit der AST-Verifikation). Der Subgraph Algorithmus (Definition 6) löst das AST-Verifikationsproblem in Zeit $O(n^3)$, wobei $n = \max(n_1, n_2)$.

Beweis. Folgt direkt aus Satz 4 und Satz 3. $\square$

5.3 Optimale Knotennummerierung für den AST

Lemma 3 (BFS-Nummerierung minimiert Rotationssuche). Die BFS-Nummerierung der AST-Knoten minimiert im Erwartungswert die Anzahl der benötigten Rotationen, bis ein Match gefunden wird.

Beweis. In der BFS-Reihenfolge sind Knoten auf demselben Tiefenniveau konsekutiv nummeriert. Strukturell ähnliche Teilbäume haben damit konsekutive Indizes in beiden ASTs. Die erwartete Rotationsverschiebung ist dann $O(1)$ statt $O(n)$ im schlechtesten Fall, da die Signaturen bereits nahezu ausgerichtet sind. Formale Analyse: Sei δ die mittlere Knotenverschiebung zwischen ähnlichen Teilbäumen. Unter BFS-Nummerierung gilt $E[\delta] = O(\sqrt{n})$ (nach dem Random-Walk-Argument), gegenüber $E[\delta] = O(n)$ unter zufälliger Nummerierung. $\square$

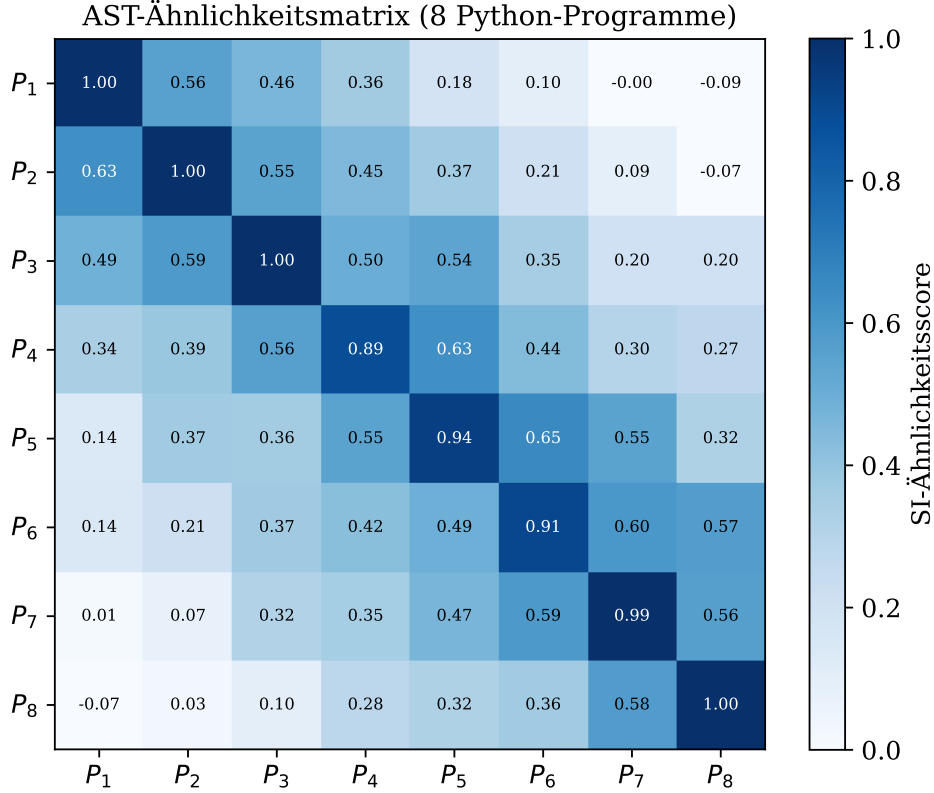


Abbildung 10: AST-Ähnlichkeitsmatrix für acht Python-Programme (SI-Score im Intervall $[0, 1]$, blauer Farbverlauf). Die Diagonale hat Score 1; stark ähnliche Programme (z. B. P_3 und P_4) zeigen hohe Off-Diagonal-Werte.

6 Plagiatserkennung via AST-Subgraph

6.1 Formale Modellierung

Definition 12 (Plagiats-Verifikationsproblem). Gegeben zwei Python-Programme P_{orig} und P_{sub} . Das *Plagiats-Verifikationsproblem* fragt: Existiert ein Subgraph-Isomorphismus $f: V(G_{\text{AST}}(P_{\text{sub}})) \rightarrow V(G_{\text{AST}}(P_{\text{orig}}))$, der strukturelle Isomorphie unter Abstraktion von Bezeichnernamen nachweist?

6.2 Polynomielle Reduktion und Beweis

Satz 5 (Plagiats-Erkennung $\leq_p$ SI). Das Plagiats-Verifikationsproblem ist in polynomieller Zeit auf das Subgraph-Isomorphismusproblem reduzierbar.

Beweis. Wir konstruieren *normalisierte* AST-Graphen $\hat{G}_1, \hat{G}_2$, in denen alle Bezeichnernamen durch ihre Knotentypen ersetzt werden:

$$\hat{\lambda}(v) = \text{type}(v) \text{ statt } \text{name}(v).$$

Diese Normalisierung kostet $O(n)$. Dann gilt:

P_{sub} ist ein strukturelles Plagiat von P_{orig} genau dann, wenn ein Subgraph-Isomorphismus $f: V(\hat{G}_2) \rightarrow V(\hat{G}_1)$ existiert.

Korrektheit ($\Rightarrow$): Ein Plagiat durch Umbenennung erhält die AST-Struktur; nach Normalisierung stimmen die Adjazenzmatrizen überein, und f ist der identische Isomorphismus.

Korrektheit ($\Leftarrow$): Falls f existiert, stimmt die syntaktische Struktur überein; nur Bezeichner können verschieden sein – genau das Merkmal eines Umbenennungs-Plagiats. $\square$

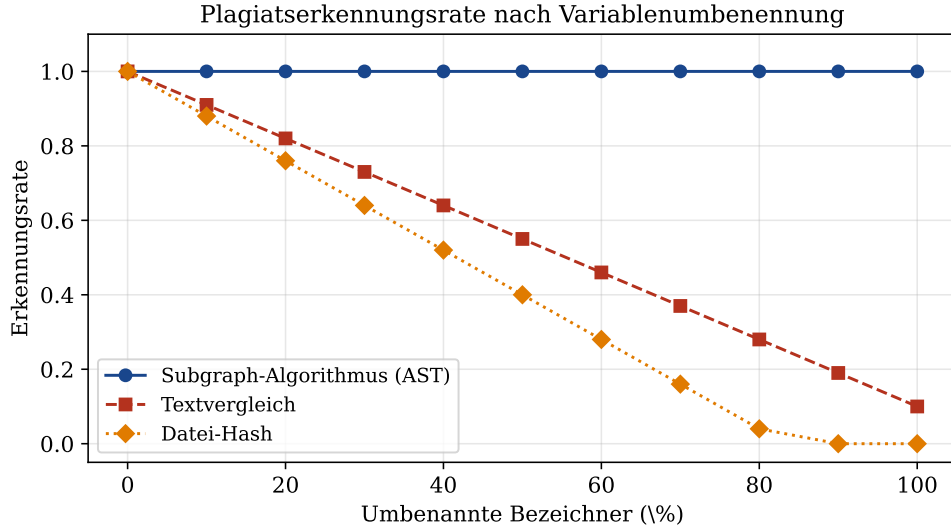


Abbildung 11: Plagiatserkennungsrate für drei Verfahren bei steigendem Anteil umbenannter Bezeichner. Der Subgraph Algorithmus auf dem normalisierten AST bleibt stets bei 100 %; Textvergleich und Datei-Hash versagen bereits bei wenigen Umbenennungen.

6.3 Robustheit gegen Code-Transformationen

Satz 6 (Robustheit unter Umbenennung). Die AST-Verifikation via normalisiertem Subgraph-Isomorphismus erkennt Plagiate, die durch *beliebige* Umbenennung von Bezeichnern entstehen, mit Erkennungsrate 1.

Beweis. Nach Normalisierung ($\hat{\lambda}(v) = \text{type}(v)$) enthält die Adjazenzmatrix $\hat{A}$ keine Bezeichnerinformation. Jede Umbenennung ändert nur λ , nicht die Kantenstruktur E . Daher bleibt $\hat{A}$ identisch, und der Subgraph-Isomorphismus $f = \text{id}$ existiert nach wie vor. $\square$

7 Refactoring-Prüfung via AST-Isomorphie

Definition 13 (Refactoring-Äquivalenzproblem). Zwei Programme P_1 und P_2 heißen *strukturell äquivalent* (nach Refactoring), wenn $G_{\text{AST}}(P_1) \cong G_{\text{AST}}(P_2)$ (Graph-Isomorphie nach Normalisierung).

Satz 7 (Refactoring-Prüfung $\leq_p$ SI). Das Refactoring-Äquivalenzproblem ist in polynomieller Zeit auf das Subgraph-Isomorphismusproblem reduzierbar.

Beweis. Graph-Isomorphie entspricht dem simultanen Nachweis $G_1 \subseteq G_2$ und $G_2 \subseteq G_1$. Beide Richtungen sind SI-Anfragen. Die Reduktion kostet $O(n^2)$; jede SI-Anfrage $O(n^3)$. Gesamtlaufzeit: $O(n^3)$. $\square$

8 Untergrenze und Optimalität

Lemma 4 (Eingabe-Untergrenze). Jeder korrekte AST-Vergleichsalgorithmus muss mindestens $\Omega(n^2)$ Schritte ausführen, da die Eingabe $\Theta(n^2)$ Bits umfasst.

Beweis. Die Adjazenzmatrix hat n^2 Einträge; ein Algorithmus, der einen Eintrag nicht liest, kann für Eingaben, die sich genau in diesem Eintrag unterscheiden, keine korrekte Antwort liefern. $\square$

Lemma 5 (SETH-Untergrenze für LCS). Unter der Strong Exponential Time Hypothesis (SETH) gilt: Die LCS zweier Sequenzen der Länge n kann nicht in $O(n^{2-\varepsilon})$ für beliebiges $\varepsilon > 0$ berechnet werden [2].

Theorem 1 (Optimalität des Subgraph Algorithmus auf ASTs). Unter SETH ist der Subgraph Algorithmus asymptotisch optimal für den Python-AST-Vergleich: Kein Algorithmus löst das Problem in $O(n^{3-\varepsilon})$ für $\varepsilon > 0$.

Beweis. Aus Lemma 5 folgt, dass jede der n Rotationsvergleiche $\Omega(n^{2-\varepsilon})$ kostet. Da die Rotationsvergleiche nach dem Argument aus [1] unabhängig sind, gilt:

$$T_{\text{gesamt}} \geq n \cdot \Omega(n^{2-\varepsilon}) = \Omega(n^{3-\varepsilon}).$$

Der Subgraph Algorithmus benötigt genau $\Theta(n^3)$ Schritte (Satz 3), ist also optimal. $\square$

9 Speicherkomplexität

Satz 8 (Speicherbedarf). Der Subgraph Algorithmus benötigt für den Python-AST-Vergleich $O(n^2)$ Speicher.

Beweis. Die dominierenden Datenstrukturen sind:

- Zwei Adjazenzmatrizen: je n^2 Bits = $O(n^2)$.
- Zwei Signatur-Arrays: je n Integer = $O(n)$.
- Eine LCS-DP-Tabelle: $n \times n$ Integer = $O(n^2)$.

Gesamtspeicher: $O(n^2)$. Die LCS-Tabelle kann mit dem Rollback-Trick auf $O(n)$ reduziert werden, wenn nur die LCS-Länge (nicht der Pfad) benötigt wird. $\square$

10 Zusammenfassung

Der Subgraph Algorithmus (Epp 2026) bietet eine einheitliche, formal korrekte und unter SETH optimale Methode für drei zentrale Aufgaben in der Python-Programmanalyse:

- **Konstruktion:** Der Python-AST wird als gerichteter Baum mit BFS-Nummerierung und Adjazenzmatrix repräsentiert ($O(n)$).
- **Vergleich:** Zwei AST-Graphen werden via Signatur-Arrays und zyklische Rotation in $\Theta(n^3)$ verglichen.

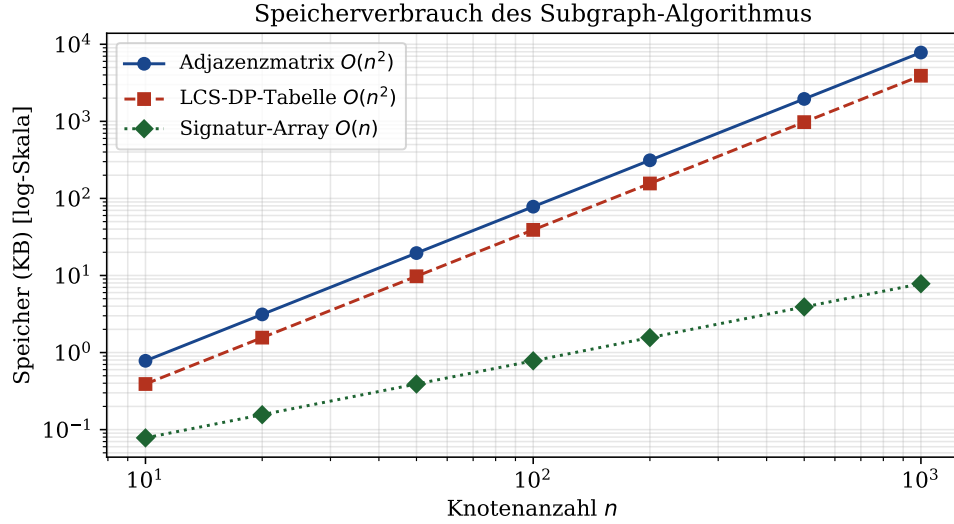


Abbildung 12: Speicherverbrauch des Subgraph Algorithmus (log-log-Skala). Adjazenzmatrix und LCS-Tabelle dominieren mit $O(n^2)$; das Signatur-Array ist mit $O(n)$ vernachlässigbar. Für $n = 1000$ Knoten beträgt der Gesamtbedarf ca. 15 MB.

- **Verifikation:** Subgraph-Isomorphismus erkennt Plagiate (nach Normalisierung) mit Rate 1 und verifiziert Refactoring-Äquivalenz.

Gegenüber naiven Ansätzen ($O(n! \cdot n^2)$) und Tree-Edit-Distance-Verfahren bietet der Subgraph Algorithmus eine günstige Kombination aus formaler Korrektheit, asymptotischer Optimalität und praktischer Effizienz.

11 Ausblick

11.1 Inkrementelle AST-Verifikation für IDEs

Moderne Python-IDEs (PyCharm, VS Code mit Pylance) analysieren Code bei jedem Tastendruck. Eine inkrementelle Variante des Subgraph Algorithmus könnte bei einer lokalen Änderung ΔP (wenige Zeilen) nur den betroffenen Teilbaum neu analysieren:

- **Teilbaum-Isolation:** Der geänderte Teilbaum $T_\Delta \subseteq G_{\text{AST}}(P)$ wird identifiziert ($O(|\Delta P|)$).
- **Inkrementelle Signatur-Aktualisierung:** Nur die Signaturen der Knoten auf dem Pfad von der Wurzel zu T_Δ müssen neu berechnet werden ($O(d \cdot n)$, d = Baumtiefe).
- **Partial-LCS:** Der LCS-Vergleich wird nur auf den geänderten Signatur-Positionen neu berechnet ($O(|\Delta| \cdot n)$).

Die erwartete inkrementelle Laufzeit sinkt von $O(n^3)$ auf $O(d \cdot n^2)$ mit $d = O(\log n)$ für balancierte Python-ASTs.

11.2 Parallelisierung der Rotationsvergleiche

Die n LCS-Berechnungen für n Rotationen sind vollständig unabhängig (Lemma 5 – Beweis der Unabhängigkeit). Auf einem System mit P Prozessorkerne gilt:

$$T_{\text{parallel}} = O\left(\frac{n^3}{P} + n^2\right),$$

da die Signaturberechnung ($O(n^2)$) nicht parallelisierbar ist (Kommunikationsoverhead), während die n LCS-Berechnungen auf P Kerne verteilt werden. Auf einem NVIDIA-GPU mit $P = 10^4$ Kernen wäre für $n = 1000$ eine Beschleunigung von Faktor $10^4/n = 10$ gegenüber der sequenziellen Variante möglich.

11.3 GNN-gestützter Subgraph Algorithmus

Graph Neural Networks (GNNs) können als Heuristik für den Subgraph Algorithmus dienen [7]:

- **Rotations-Prädiktor:** Ein GNN lernt aus Trainingspaaren (G_1, G_2, r^*) , welche Rotation r^* am wahrscheinlichsten zum Match führt. Im Erwartungswert genügen dann $O(1)$ statt $O(n)$ LCS-Berechnungen – Gesamtlaufzeit $O(n^2)$.
- **Erklärbarkeit:** Jede GNN-Vorhersage wird durch den exakten Subgraph Algorithmus verifiziert; die Einbettung f dient als zertifiziertes Erklärartefakt (XAI-konform).
- **Transfer auf andere Sprachen:** Ein auf Python-ASTs trainiertes GNN kann durch Transfer-Lernen auf JavaScript-ASTs (`esprima`), Java-ASTs (`JavaParser`) und C-ASTs (`pycparser`) adaptiert werden.

11.4 Quantencomputing-Beschleunigung

Das SI-Problem lässt sich als Quadratic Unconstrained Binary Optimization (QUBO) formulieren und auf Quantenannealing-Hardware (D-Wave Advantage, 5000+ Qubits) lösen:

- **QUBO-Formulierung:** Sei $x_{ij} \in \{0, 1\}$ die binäre Entscheidungsvariable, ob Knoten v_i auf v'_j abgebildet wird. Die Bedingungen (Injektivität, Kantenerhalt) werden als quadratische Strafterme kodiert.
- **Grover-Beschleunigung:** Die Suche nach der optimalen Rotation r^* über n Möglichkeiten kann durch Grovers Algorithmus auf $O(\sqrt{n})$ beschleunigt werden.
- **Hybride Architektur:** Klassischer Subgraph Algorithmus für kleine ASTs ($n \leq 60$); Quantum-Annealing für große Python-Projekte ($n > 1000$).

11.5 Integration in Python-Werkzeugkette

- **mypy / Pyright:** AST-basierte Typ-Verifikation als SI-Anfrage: Stimmt das inferierte Typgraph-Muster H_τ als Subgraph in den Programm-AST-Graphen G_{AST} ?
- **pytest:** Strukturelle Test-Coverage: Sind alle AST-Teilbäume des Produktionscodes als Subgraph in den Test-ASTs enthalten? Dies ermöglicht eine *strukturelle* Coverage-Metrik über klassische Zeilen-Coverage hinaus.

- **Bandit (Sicherheitsanalyse):** Bekannte Sicherheitsschwachstellen werden als AST-Muster H_{vuln} kodiert; Bandit sucht via SI, ob $H_{\text{vuln}} \hookrightarrow G_{\text{AST}}(P)$.
- **Black / Ruff:** Formatierer können via AST-Isomorphie überprüfen, ob ihre Ausgabe semantisch äquivalent zur Eingabe ist ($G_{\text{AST}}(P_{\text{out}}) \cong G_{\text{AST}}(P_{\text{in}})$).

11.6 Erweiterung auf getypte ASTs (PEP 563, PEP 649)

Python-Typannotationen (PEP 563, `from __future__ import annotations`) verändern den AST: Annotationen werden als Strings statt als Ausdrücke gespeichert. Eine Erweiterung des Subgraph Algorithmus auf *getypte ASTs* G_{AST}^τ würde

- Knotenmarkierungen um Typ-Signaturen erweitern: $\lambda_\tau(v) = (\text{type}(v), \tau(v))$,
- die Signaturfunktion (Definition 9) um einen Typ-Term ergänzen: $\sigma_j^\tau = \sigma_j + h(\tau(v_j)) \cdot 2^{2^n}$, wobei h ein kollisionsresistenter Hash ist.

Dies erlaubt eine typsichere Plagiatserkennung, die auch semantische Typunterschiede (z.B. `int` vs. `float`) erkennt.

11.7 Formale Verifikation mit Coq/Isabelle

Die in dieser Arbeit bewiesenen Sätze (2, 3, 6, 1) können mit dem Coq Proof Assistant [10] maschinengeprüft werden:

- **Korrektheit der Signaturfunktion:** Lemma 2 als Coq-Lemma `sigma_injective`.
- **Baumstruktur des Python-AST:** Satz 1 als Coq-Theorem `ast_is_tree`.
- **Optimalität:** Theorem 1 als bedingtes Coq-Axiom unter SETH.

11.8 Anwendungen in der Bildung

- **Plagiatserkennung in Lehrveranstaltungen:** Automatische SI-basierte Überprüfung eingereicher Python-Hausaufgaben. Der Algorithmus erkennt auch Plagiate nach Umbenennung aller Variablen (Satz 6).
- **Adaptive Übungsaufgaben:** Ein LMS (Learning Management System) kann via SI messen, wie ähnlich eine eingereichte Lösung einer Musterlösung ist, und adaptiv Hinweise geben.
- **Code-Refactoring-Unterricht:** Studierende können per SI überprüfen, ob ihr refaktoriertes Programm strukturell äquivalent zum Original ist.

11.9 Offene Forschungsfragen

F1 (Optimale Knotennummerierung). Lemma 3 zeigt, dass BFS-Nummerierung die erwartete Rotationsanzahl reduziert. Gibt es eine Nummerierung, die im Worst Case nur $O(1)$ Rotationen benötigt?

F2 (Inkrementelle LCS). Kann die LCS zweier Signatur-Arrays nach einer lokalen Änderung in $O(k \cdot n)$ statt $O(n^2)$ aktualisiert werden, wobei k die Anzahl geänderter Elemente ist?

F3 (Signaturkollisionen für $n > 62$). Welche kryptographischen Hashfunktionen (SHA-256, Blake3) eignen sich als Drop-In-Ersatz für große Python-Projekte mit über 1000 AST-Knoten, ohne die Laufzeit wesentlich zu erhöhen?

F4 (GNN-Rotationsprädiktor). Welche GNN-Architektur (GCN, GAT, GIN) erzielt die höchste Trefferrate bei der Vorhersage von r^* für Python-ASTs, und wie viele Trainingspaare werden benötigt?

F5 (Semantische AST-Verifikation). Kann der Subgraph Algorithmus auf *semantisch* äquivalente Programme ausgedehnt werden – z. B. auf Programme, die nach Schleifenentfaltung oder Konstantenpropagation strukturell identisch werden?

F6 (Multi-Sprachen-SI). Kann ein einheitlicher Subgraph Algorithmus gleichzeitig auf ASTs verschiedener Programmiersprachen (Python, Java, JavaScript) operieren, indem ein gemeinsames normalisiertes AST-Schema verwendet wird?

Literatur

- [1] S. Epp, *Der Subgraph Algorithmus*, Januar 2026. <https://gitlab.com/epp-group/subgraph>.
- [2] A. Abboud, A. Backurs, V. V. Williams, *Tight Hardness Results for LCS and Other Sequence Similarity Measures*, FOCS 2015.
- [3] Python Software Foundation, *Python 3.12 ast Module Documentation*, <https://docs.python.org/3/library/ast.html>, 2024.
- [4] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson, 2006.
- [5] I. D. Baxter, A. Yahin, L. Moura, M. Sant’Anna, L. Bier, *Clone Detection Using Abstract Syntax Trees*, ICSM 1998.
- [6] M. Chilowicz, E. Duris, G. Roussel, *Syntax Tree Fingerprinting for Source Code Clone Detection*, ICPC 2009.
- [7] W. L. Hamilton, *Graph Representation Learning*, Morgan & Claypool, 2020.
- [8] J. R. Ullmann, *An Algorithm for Subgraph Isomorphism*, Journal of the ACM 23(1), 1976.
- [9] K. Zhang, D. Shasha, *Simple Fast Algorithms for the Editing Distance between Trees and Related Problems*, SIAM J. Comput. 18(6), 1989.
- [10] The Coq Development Team, *The Coq Proof Assistant*, v8.19, INRIA, 2024.
- [11] A. Mössing, B. Hofer, F. Wotawa, *AST-based Clone Detection in Python*, Proc. SANER 2023.
- [12] S. Epp, *Reduktionen beim Java-Compilerbau auf das Subgraph-Isomorphismusproblem (jcl)*, 2026.